

Playwright

From Zero to Viva-Ready

Modern web automation — auto-waiting, locators, fixtures, trace viewer, API testing + 25 viva questions with answers

Quick-scan guide for SQA interviews and vivas

Companion to: [API Testing Guide](#) · [Selenium Guide](#) · [JMeter Guide](#) · [CI/CD Guide](#) · [SQA A-Z Guide](#)

PART 1 — What is Playwright?

Playwright is a **modern browser automation framework by Microsoft** (built by the team behind Puppeteer). Like Selenium, it controls browsers with code — but it was designed 15 years later, so it fixes the classic pains: it **auto-waits** for elements (almost no manual waits), controls the browser over a fast persistent connection instead of one-HTTP-call-per-command, and ships everything in one install: test runner, assertions, parallel execution, reports, screenshots, video, and a time-travel debugger (Trace Viewer).

	Selenium	Playwright
Waiting	You add explicit waits	Auto-waits before every action
Setup	Library + separate driver + separate runner (TestNG)	One npm install — runner included
Browsers	Real Chrome/Firefox/Edge/Safari via drivers	Chromium, Firefox, WebKit (Safari engine) bundled
Speed/flakiness	Slower, flaky if waits are wrong	Faster, far less flaky by design
Languages	Java, Python, C#, JS, Ruby...	JS/TypeScript (primary), Python, Java, .NET
Extras	Via third-party tools	Built-in: trace viewer, codegen, video, network mocking, API testing

One-line viva answer: "Playwright is a modern, auto-waiting, batteries-included alternative to Selenium, best known for speed and low flakiness."

PART 2 — Core concepts

Browser → Context → Page (the hierarchy)

- **Browser**: the running browser instance.
- **BrowserContext**: an isolated incognito-like profile inside the browser — own cookies, storage, login state. Contexts are cheap, so every test gets a fresh one = perfect test isolation without relaunching the browser.
- **Page**: one tab inside a context.

This hierarchy is why Playwright tests are fast AND isolated — a favourite viva differentiator.

Auto-waiting (the killer feature)

Before every click/fill, Playwright automatically waits until the element is: attached, visible, stable (not animating), enabled, and able to receive events. So `page.click()` already contains what Selenium needs an explicit wait for. Web-first assertions retry too: `expect(locator).toHaveText('Done')` keeps re-checking until timeout.

Locators (user-facing first)

Playwright pushes you to locate things the way a USER sees them, which survives DOM refactors:

Locator	Example
<code>getByRole</code> (best)	<code>page.getByRole('button', { name: 'Login' })</code>
<code>getByLabel</code>	<code>page.getByLabel('Email address')</code>
<code>getByPlaceholder</code>	<code>page.getByPlaceholder('Search...')</code>
<code>getByText</code>	<code>page.getByText('Welcome back')</code>
<code>getByTestId</code>	<code>page.getByTestId('submit-btn') // data-testid attribute</code>

Locator	Example
CSS / XPath	page.locator('#login') — supported, but last resort

PART 3 — A real test + the toolbox

```
import { test, expect } from '@playwright/test';

test('user can log in', async ({ page }) => {
  await page.goto('https://site.com/login');
  await page.getByLabel('Email').fill('abrar@test.com');
  await page.getByLabel('Password').fill('secret123');
  await page.getByRole('button', { name: 'Login' }).click();
  await expect(page.getByText('Welcome')).toBeVisible();
});
```

Read it aloud: go to the page, fill email, fill password, click Login, expect Welcome to be visible. No waits written — they're all automatic. The { page } in brackets is a **fixture**: Playwright hands each test a fresh page in a fresh context.

Tool	What it does
playwright.config.ts	Central config: browsers (projects), baseURL, retries, parallel workers, reporters, trace/video settings
Projects	Run the same tests on chromium + firefox + webkit (and mobile viewports) by listing them in config
Codegen	npx playwright codegen site.com — records your clicks and writes test code for you (great starting point)
Trace Viewer	Time-travel debugger: screenshots of every step, DOM snapshots, network, console — opens with npx playwright show-trace
UI Mode	npx playwright test --ui — watch tests run step by step interactively
HTML report	Built-in report with failures, screenshots, videos: npx playwright show-report
Fixtures & hooks	test.beforeEach / custom fixtures replace setup-teardown boilerplate; storageState reuses login across tests
Network mocking	page.route() intercepts requests — fake a 500 error or stub an API without touching the backend
API testing	Built-in request fixture sends pure HTTP calls — mix API setup (create data fast) with UI verification
Parallelism & retries	Tests run in parallel workers by default; retries: 2 in CI marks flaky tests instead of failing builds

Interview gold: "I create test data via the API (fast, reliable), then verify it in the UI — Playwright lets me do both in the same test." This connects directly to my Postman/API portfolio project.

PART 4 — 25 Viva Questions with Answers

Q: What is Playwright?

A: A modern open-source browser automation framework by Microsoft with auto-waiting, a built-in test runner, and support for Chromium, Firefox, and WebKit.

Q: Playwright vs Selenium?

A: Playwright auto-waits (less flaky), ships runner+assertions+reports in one install, uses a fast persistent connection, and includes trace viewer/codegen/mocking. Selenium is more mature, supports more

languages, and drives real branded browsers via the W3C standard.

Q: Which browsers does it support?

A: Chromium (Chrome/Edge), Firefox, and WebKit — the Safari engine, which is how you test Safari behavior on Windows/Linux.

Q: What is auto-waiting?

A: Before any action, Playwright waits until the element is attached, visible, stable, enabled, and can receive events — no manual waits for normal flows.

Q: What are web-first assertions?

A: expect() assertions that automatically RETRY until they pass or time out — e.g. toBeVisible, toHaveText. They absorb timing issues instead of failing instantly.

Q: What is a BrowserContext?

A: An isolated incognito-like session inside one browser — separate cookies/storage. Each test gets a fresh context: full isolation without the cost of relaunching the browser.

Q: Browser vs context vs page?

A: Browser = the program; context = an isolated profile inside it; page = a tab inside a context.

Q: Which locator is recommended and why?

A: getByRole with an accessible name — it finds elements the way users and screen readers do, so it survives DOM/CSS refactors. Then getByLabel/getByText/getByTestId; raw CSS/XPath last.

Q: What is getByTestId?

A: Locates by the data-testid attribute developers add specifically for tests — stable by agreement between dev and QA.

Q: What is a fixture?

A: Reusable setup injected into tests via arguments — { page } gives every test a fresh page. You can define custom fixtures (e.g. a logged-in page) to kill boilerplate.

Q: How do you handle login across many tests?

A: Log in once, save storageState to a file, and load it in config — every test starts already authenticated. Massive speed win.

Q: What is the Trace Viewer?

A: A time-travel debugger: records every step with before/after screenshots, DOM snapshots, network calls, and console logs. When CI fails, I download the trace and replay exactly what happened.

Q: What is codegen?

A: npx playwright codegen URL records your manual actions and generates test code with good locators — a starting draft you then refactor.

Q: How does Playwright run tests in parallel?

A: The runner spawns multiple worker processes by default; test files run in parallel. Config controls worker count; fullyParallel spreads even tests within a file.

Q: What are projects in the config?

A: Named configurations — chromium, firefox, webkit, mobile viewports — the same suite runs once per project for cross-browser coverage.

Q: How do you handle flaky tests?

A: retries: 2 in CI re-runs failures and flags them as 'flaky' instead of failing the build; then I fix the root cause using the trace. Auto-waiting prevents most flakiness up front.

Q: Can Playwright test APIs?

A: Yes — the built-in request fixture makes pure HTTP calls with assertions. Common pattern: create test data via API, verify via UI.

Q: What is `page.route()`?

A: Network interception — block, modify, or stub any request. I can force a 500 error or return fake JSON to test error handling without touching the backend.

Q: How do you take screenshots/videos?

A: Config: screenshot: 'only-on-failure', video: 'retain-on-failure', trace: 'on-first-retry' — evidence is captured automatically exactly when needed.

Q: What is expect polling/timeout?

A: Assertions retry up to a timeout (default 5 s, configurable). Different from action timeout (30 s default) — knowing the difference is a senior signal.

Q: How do you do POM in Playwright?

A: Same idea as Selenium: a class per page holding locators + actions, instantiated with the page fixture. Locators defined once in the constructor.

Q: How do you run Playwright in CI?

A: `npx playwright install --with-deps` to get browsers, then `npx playwright test`; upload the HTML report and traces as artifacts. Official GitHub Actions template exists — same CI concepts as my Newman pipeline.

Q: What is UI Mode?

A: `npx playwright test --ui` — an interactive watch mode showing each step live with time-travel, great for local debugging.

Q: Hard waits in Playwright?

A: `page.waitForTimeout()` exists but is a code smell — auto-waiting and web-first assertions should make it unnecessary.

Q: Why would a team choose Playwright today?

A: Speed, isolation via contexts, near-zero flakiness, one-package setup, first-class TypeScript, and superb debugging (trace viewer). The main trade-offs: newer ecosystem than Selenium and engine-based (not branded-browser) testing.

Last-minute quick scan

If they say...	You say...
Playwright?	Microsoft's modern auto-waiting browser automation with built-in runner.
Killer feature?	Auto-waiting + retrying assertions = flakiness designed away.
Browsers?	Chromium, Firefox, WebKit — WebKit gives Safari coverage anywhere.
Isolation?	Fresh <code>BrowserContext</code> per test — incognito-cheap, not relaunch-expensive.
Best locator?	<code>getByRole</code> — find things like a user would.
Debugging?	Trace Viewer = time travel; UI mode for local; codegen to draft tests.
Login reuse?	<code>storageState</code> — authenticate once, reuse everywhere.
Mock network?	<code>page.route()</code> — stub APIs, force errors.

Bridge line for viva: "My repo demonstrates the API layer with Postman/Newman; Playwright is how I'd cover the UI layer — and its request fixture even lets me reuse my API skills inside UI tests."